

L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765



MECHTRON 3TB4

Lab 4 Report

Date: March 27, 2026

Lab 4 Report

Overview

The goal of Lab 4 is to design and test a system that enables the Nios V processor to communicate with the external SDRAM on the DE1-SoC FPGA board. This lab focuses on the hardware and software codesign using an FPGA SOPC (System-on-a-Programmable-Chip). It provides insight into how hardware modules, memory controllers, and software interact and integrate to form a complete system.

In the lab, an SDRAM controller interface was created and integrated into a Nios V-based system using Platform Designer in Quartus. The controller enables the processor to access the SDRAM via the Avalon Memory-Mapped Interface. The Avalon Memory-Mapped Interface is used to communicate with the processor, since SDRAM devices have complex timing requirements, making it easier to use a pre-designed controller. A wrapper is created to connect the controller to the system.

Using embedded C, software is developed in Ashling RiscFree IDE to test the functionality of the SDRAM controller. The software writes and reads the different data types, such as char, short, and int, to test the functionality of the memory operations. Furthermore, the Signal Tap Logic Analyzer is used to observe the internal signals while the controller is running.

In conclusion, the lab aims to demonstrate the functionality of the combined operation of the hardware development tools for FPGAs and the embedded software development tools.

Questions

1. Using screenshots taken in the “Develop Software” part of the lab, explain the following:
 - a) Why does it appear that writing takes fewer clock cycles than reading?

It appears that writing takes fewer clock cycles than reading, since writing does not require retrieving data from memory, unlike read operations. In this case, reading data from the SDRAM takes more time, whereas writing data to memory has no associated retrieval time, as shown in the figure below.

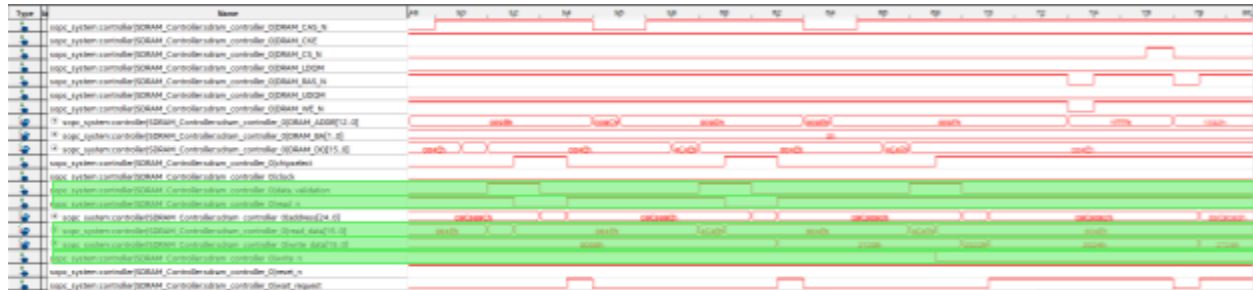


Figure 1. Screenshot of the Signal Tap Logic Analyzer for Q1a.

- b) While reading or writing a char value, e.g. 5, why does the SDRAM DQ show values such as 0x0505 instead of 0x0005?

A char value consists of 8 bits (1 byte), while the SDRAM data bus is 16 bits (2 bytes) wide. Therefore, the 8-bit char value is duplicated across both bytes, which means that 0x0005 becomes 0x0505. Both the bytes appear on the data bus, but there are byte enable signals, LDQM and UDQM, that determine which byte is actually written or read. For our case, the values 0x004E, 0x4C4E are 16-bit patterns instead of the single 8-bit value such as 0x0005 as shown in the figure below.

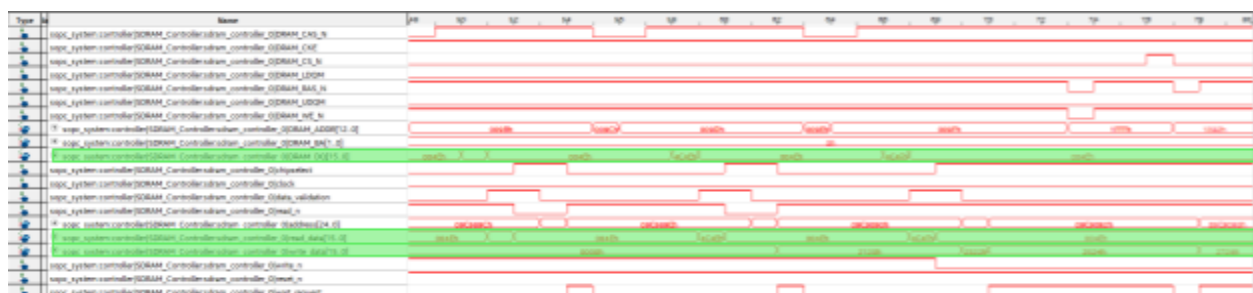


Figure 2. Screenshot of the Signal Tap Logic Analyzer for Q1b.

- c) While reading or writing two consecutive char values, e.g. 5 and 6 using memory locations 0x0000 and 0x0001, why does the SDRAM ADDR show the same 0x0000 value for both locations?

SDRAM uses word addressing, where each word is 2 bytes (16 bits). Therefore, both of the memory locations 0x0000 and 0x0001 map to the same word of 0x0000, which is why the SDRAM ADDR shows the same 0x0000 value for both locations.



Figure 3. Screenshot of the Signal Tap Logic Analyzer Q1c.

2. What steps does the Avalon interconnect take to write a 32-bit integer into the 16-bit SDRAM memory?

The NIOSV processor sends a 32-bit write request through the Avalon bus, whereas the SDRAM controller uses a 16-bit data width. This is achieved when the Avalon bus automatically splits the 32-bit data into two 16-bit transactions. The lower 16 bits are written to the base address, whereas the upper 16 bits are written to the next address offset. Byte enable signals are used to ensure the correct bytes are written.

- Open the compilation report in Quartus, and report the following numbers

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 20 21:35:13 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab4
Top-level Entity Name	lab4
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	4,370 / 32,070 (14 %)
Total registers	5768
Total pins	57 / 457 (12 %)
Total virtual pins	0
Total block memory bits	1,463,424 / 4,065,280 (36 %)
Total DSP Blocks	3 / 87 (3 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	0 / 4 (0 %)

Figure 4. Compilation report in Lab 4.

Table 1. Summary for Components for the compilation report in Lab 4.

Feature	Quantity
Total number of logic elements	4370
Total number of registers	5768
Total number of pins	57
Max number of logic elements that can fit on FPGA	32,070

4. Considering just the maximum number of logic elements on the FPGA, approximately how many SOPC systems like the one you built could fit on the FPGA?

$$\frac{\text{Total Logic Elements}}{\text{Used Logic Elements}} = \frac{32,070}{4370} = 7.3 = 7 \text{ (rounded)}$$

Therefore, considering just the maximum number of logic elements on the FPGA, approximately 7 SOPC systems could fit on the FPGA.

5. Considering just the maximum amount of memory available in the FPGA, approximately how many SOPC systems like the one you built could fit on the FPGA?

The FPGA has a max memory of 4,065,280 bits, whereas the SOPC utilized 172,032 bytes (programmed this way for the bonus question, but didn't finish it at the end) for the on-chip memory. Therefore, the maximum number of systems that can fit on the FPGA is approximately 3, given the calculation below:

$$\frac{\text{total FPGA embedded memory}}{\text{memory used by one SOPC}} = \frac{4,065,280}{172,032 * 8} = 2.95 = 3 \text{ (rounded)}$$

But, inferring you cannot program a partial SOPC onto the FPGA, therefore, only 2 SOPC systems can be built on the FPGA, given our parameters.

6. Considering just the maximum number of pins on the FPGA, how many SOPC systems like the one you built could fit on the FPGA?

$$\frac{\text{Total Pins}}{\text{Used Pins}} = \frac{457}{57} = 8.02 = 8 \text{ (rounded)}$$

Therefore, considering just the maximum number of pins on the FPGA, approximately 8 SOPC systems could fit on the FPGA.

SDRAM Controller

The SDRAM Controller is in the module named SDRAM_Controller.v, which is an interface between the Nios V system and the SDRAM chip connected to the DE1-SoC board. It wraps the provided SDRAM controller and specifies the input/output ports to integrate the Avalon memory-mapped interface with the hardware side's SDRAM signals. On the Avalon side, the controller processes signals, such as the address, read/write signals, byte enable signals, data signals, etc., from the processor side. On the hardware side, the module processes the SDRAM signals such as the clock, chip enable, row address strobe, column address strobe, write enable, bank addresses, data signals, etc., to execute the appropriate command sequences to the SDRAM to execute the memory operations. The controller on the hardware side processes the SDRAM signals, such as the clock, chip enable, row address strobe, column address strobe, write enable, bank addresses, data signals, etc., to execute the appropriate command sequences to the SDRAM to execute the memory operations.

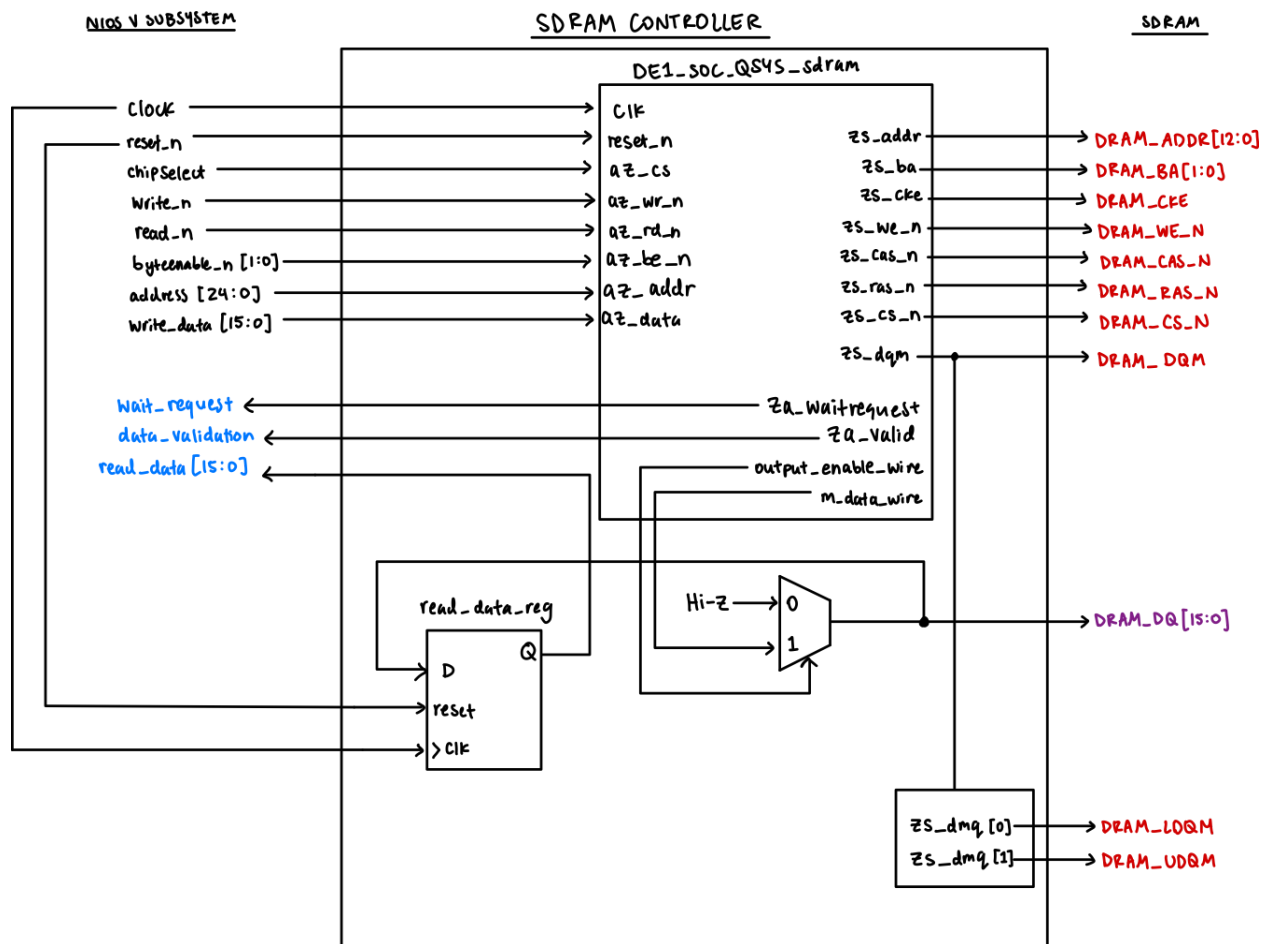


Figure 5. Drawn schematic of Circuit for SDRAM_Controller.v.

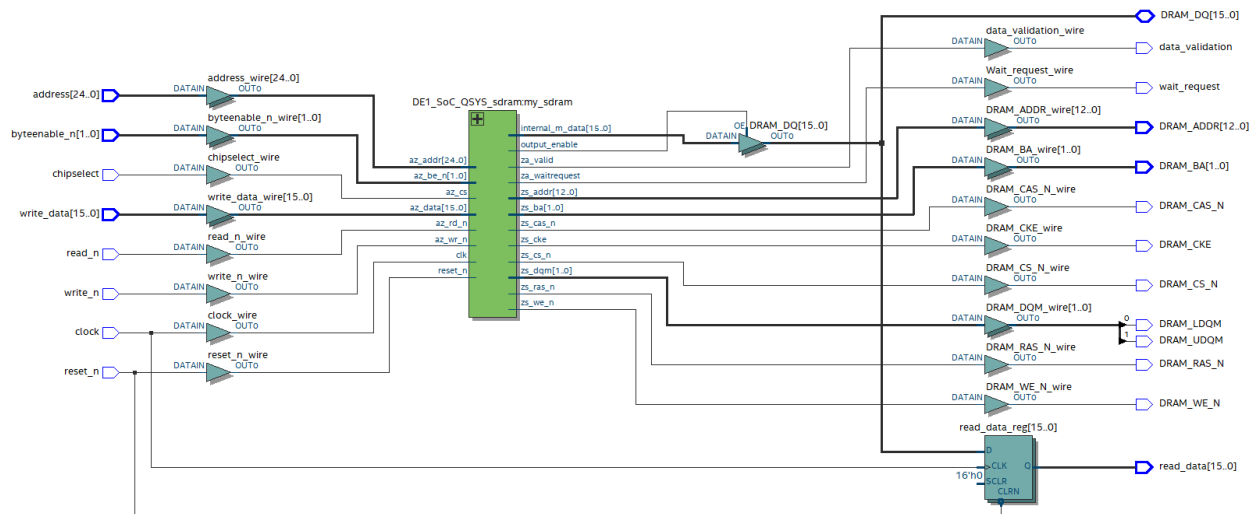


Figure 6. Schematic of Circuit for SDRAM_Controller.v.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 20 18:05:19 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab4
Top-level Entity Name	SDRAM_Controller
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	184 / 32,070 (< 1 %)
Total registers	300
Total pins	104 / 457 (23 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 7. Screenshot of the SDRAM_Controller.v module compilation report.

Table 2. Summary for Components for the SOPC System module in Lab 4.

Feature	Quantity
Total number of logic elements	184
Total number of registers	300
Total number of pins	104
Max number of logic elements that can fit on FPGA	32,070

```

1  /*****
2  *
3  * Module:      SDRAM_Controller
4  * Description:
5  * This module is used for the sram controller for MT3TB4 Lab 4
6  *
7  *****/
8  `timescale 1ns / 1ps
9
10 module SDRAM_Controller ( //the ports sequence follows the sequence of wires in the diagram in the lab manual
11 //signals through Avalon Interface:
12     input      clock,
13     input      reset_n,
14     input      chipselect,
15     input      write_n,
16     input      read_n,
17     input [1:0] byteenable_n,
18     input [24:0] address, //2's power of 25 is 33,554,432 that is 32M. (16 bits word, that makes capable for 64MBytes)
19     input [15:0] write_data,
20     output [15:0] read_data,
21     output      wait_request,
22     output      data_validation,
23
24     //signals between Controller and SDRAM:
25
26     inout [15:0] DRAM_DQ,
27     output [12:0] DRAM_ADDR,
28     output [1:0] DRAM_BA,
29     // output
30     output      DRAM_CLK,
31     output      DRAM_CKE,
32     output      DRAM_LDQM,
33     output      DRAM_UDQM,
34     output      DRAM_WE_N,
35     output      DRAM_CAS_N,

```

Figure 8. Screenshot of the SDRAM_Controller.v System module.

SOPC System

The System on a Programmable Chip (SOPC) system is generated using the Platform Designer, creating an `sopc_system.qip` file that contains all necessary Verilog HDL .v files to integrate the system components. The SOPC integrates the Intel NIOSV processor, SDRAM controller, on-chip memory, and JTAG UART into a single system on the FPGA. In the design, the NIOSV processor acts as the master device on the Avalon Memory-mapped bus that issues read and write commands. The SDRAM controller, on-chip memory, and JTAG UART act as slave devices responding to these requests.

The SDRAM controller is connected to the external SDRAM chip, which allows the processor to access a large memory space. Proper clock implementation using PLL ensures the SDRAM clock leads the system clock by 3 ns, which is required for correct SDRAM operations done by `sys_sdram_pll_0`. The components are all connected through the Avalon bus with interconnect logic, address mapping, and system integration for communications between modules automatically generated by the Platform Designer.

System: socp_system Path: clk_0										
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
✓		clk_0	Clock Source							
		clk_in	Clock Input	clk	exported					
		clk_in_reset	Reset Input	reset	clk_0					
		clk	Clock Output	Double-click to export						
		clk_reset	Reset Output	Double-click to export						
✓		sys_sdram_pll_0	System and SDRAM Clocks for D...							
		ref_clk	Clock Input	Double-click to export	clk_0					
		ref_reset	Reset Input	Double-click to export						
		sys_clk	Clock Output	Double-click to export	sys_sdra...					
		sdram_clk	Clock Output	Double-click to export	sdram_clk					
		reset_source	Reset Output	Double-click to export	sys_sdra...					
✓		intel_niosv_g_0	Nios V/g General Purpose Proce...							
		clk	Clock Input	Double-click to export	sys_sdra...					
		reset	Reset Input	Double-click to export	[clk]					
		platform_irq_rx	Interrupt Receiver	Double-click to export				IRQ 0	IRQ 15	
		instruction_manager	AXI4 Master	Double-click to export	[clk]					
		data_manager	AXI4 Master	Double-click to export	[clk]					
		timer_sw_agent	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0005_0000	0x0005_003f			
		dm_agent	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0004_0000	0x0004_ffff			
✓		onchip_memory2_0	On-Chip Memory (RAM or ROM)...							
		clk1	Clock Input	Double-click to export	sys_sdra...					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0000_0000	0x0002_9fff			
		reset1	Reset Input	Double-click to export	[clk1]					
✓		jtag_uart_0	JTAG UART Intel FPGA IP							
		clk	Clock Input	Double-click to export	sys_sdra...					
		reset	Reset Input	Double-click to export	[clk]					
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0005_0040	0x0005_0047			
		irq	Interrupt Sender	Double-click to export	[clk]					
✓		SDRAM_Controller...	SDRAM_Controller							
		reset	Reset Input	Double-click to export	[clock]					
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock]	# 0x0400_0000	0x07ff_ffff			
		conduit_end	Conduit	sdram_dq	[clock]					
		conduit_end_1	Conduit	sdram_addr	[clock]					
		conduit_end_2	Conduit	sdram_cs_n	[clock]					
		conduit_end_3	Conduit	sdram_we_n	[clock]					
		conduit_end_4	Conduit	sdram_ba	[clock]					
		conduit_end_5	Conduit	sdram_ras_n	[clock]					
		conduit_end_6	Conduit	sdram_cas_n	[clock]					
		conduit_end_7	Conduit	sdram_udqm	[clock]					
		conduit_end_8	Conduit	sdram_ldqm	[clock]					
		conduit_end_9	Conduit	sdram_cke	[clock]					
		clock	Clock Input	Double-click to export	sys_sdra...					

Figure 9. Screenshot of the Component Connections, IRQ and Export.

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Sun Mar 22 12:14:47 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab4
Top-level Entity Name	sopc_system
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	4,369 / 32,070 (14 %)
Total registers	5745
Total pins	41 / 457 (9 %)
Total virtual pins	0
Total block memory bits	1,463,424 / 4,065,280 (36 %)
Total DSP Blocks	3 / 87 (3 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	0 / 4 (0 %)

Figure 10. Compilation report of the SOPC System module.

Table 3. Summary for Components for the SOPC System module in Lab 4.

Feature	Quantity
Total number of logic elements	4369
Total number of registers	5745
Total number of pins	41
Max number of logic elements that can fit on FPGA	32,070

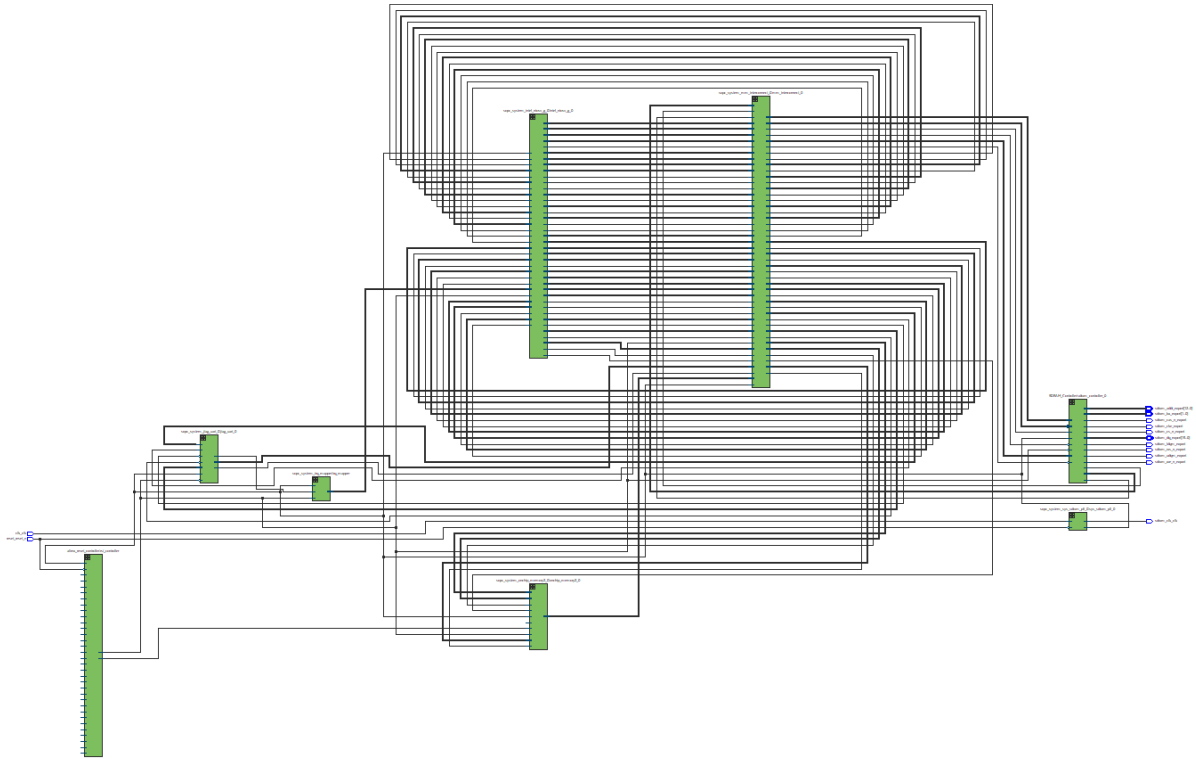


Figure 11. RTL view of the circuit

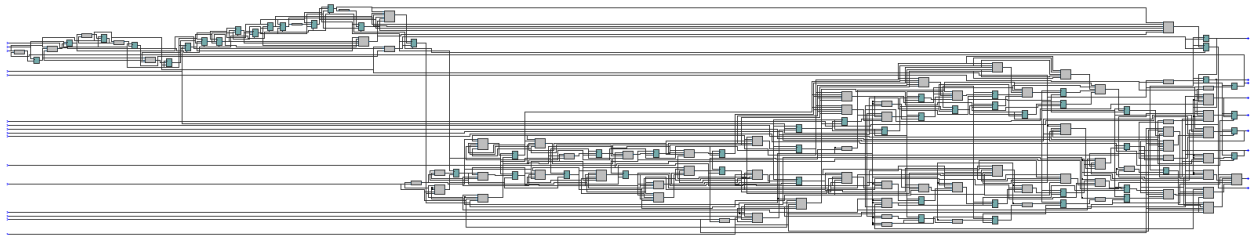


Figure 12. Technology map viewer (post-mapping) of the circuit.

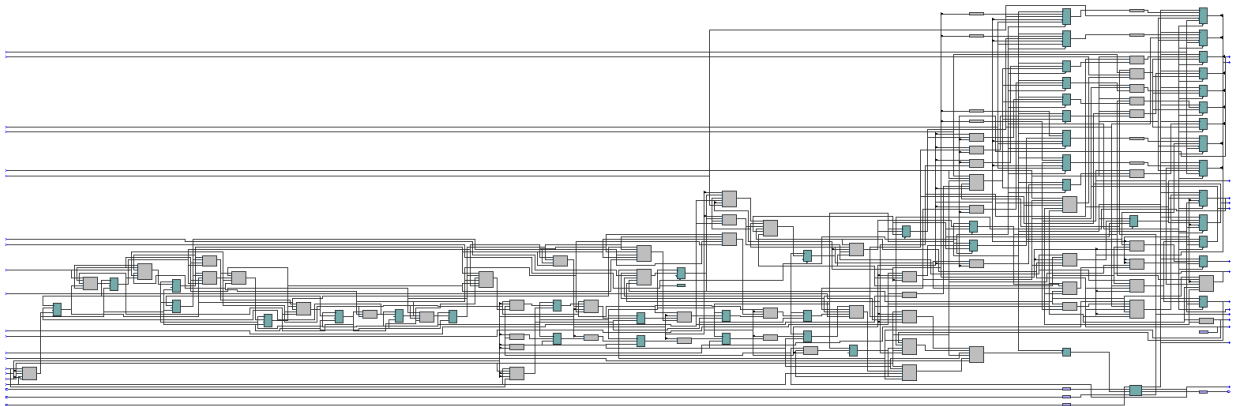


Figure 13. Technology map viewer (post-fitting) of the circuit.

```

1 // socp_system.v
2
3 // Generated using ACDS version 23.1 991
4
5 `timescale 1 ps / 1 ps
6 module socp_system (
7     input wire      clk_clk,          //      clk.clk
8     input wire      reset_reset_n,    //      reset.reset_n
9     output wire [12:0] sdram_addr_export, // sdram_addr.export
10    output wire [1:0] sdram_ba_export,   // sdram_ba.export
11    output wire      sdram_cas_n_export, // sdram_cas_n.export
12    output wire      sdram_cke_export,   // sdram_cke.export
13    output wire      sdram_clk_clk,      // sdram_clk.clk
14    output wire      sdram_cs_n_export,   // sdram_cs_n.export
15    inout wire [15:0] sdram_dq_export,    // sdram_dq.export
16    output wire      sdram_ldqm_export,   // sdram_ldqm.export
17    output wire      sdram_ras_n_export,  // sdram_ras_n.export
18    output wire      sdram_udqm_export,   // sdram_udqm.export
19    output wire      sdram_we_n_export,   // sdram_we_n.export
20 );
21
22 wire      sys_sdram_pll_0_sys_clk_clk; // sys_sdram_pll_0:sys_clk_clk -> [SDRAM_Controller_0:clock, int
23 wire [7:0] intel_niosv_g_0_data_manager_arlen; // intel_niosv_g_0:data_manager_arlen -> mm_interconnect_0:intel
24 wire [3:0] intel_niosv_g_0_data_manager_wstrb; // intel_niosv_g_0:data_manager_wstrb -> mm_interconnect_0:intel
25 wire      intel_niosv_g_0_data_manager_wready; // mm_interconnect_0:intel_niosv_g_0_data_manager_wready -> inte
26 wire      intel_niosv_g_0_data_manager_rready; // intel_niosv_g_0:data_manager_rready -> mm_interconnect_0:intel
27 wire [7:0] intel_niosv_g_0_data_manager_awlen; // intel_niosv_g_0:data_manager_awlen -> mm_interconnect_0:intel
28 wire      intel_niosv_g_0_data_manager_wvalid; // intel_niosv_g_0:data_manager_wvalid -> mm_interconnect_0:intel
29 wire [31:0] intel_niosv_g_0_data_manager_araddr; // intel_niosv_g_0:data_manager_araddr -> mm_interconnect_0:intel
30 wire [2:0] intel_niosv_g_0_data_manager_arprot; // intel_niosv_g_0:data_manager_arprot -> mm_interconnect_0:intel
31 wire [2:0] intel_niosv_g_0_data_manager_awprot; // intel_niosv_g_0:data_manager_awprot -> mm_interconnect_0:intel
32 wire [31:0] intel_niosv_g_0_data_manager_wdata; // intel_niosv_g_0:data_manager_wdata -> mm_interconnect_0:intel
33 wire      intel_niosv_g_0_data_manager_arvalid; // intel_niosv_g_0:data_manager_arvalid -> mm_interconnect_0:intel
34 wire [31:0] intel_niosv_g_0_data_manager_awaddr; // intel_niosv_g_0:data_manager_awaddr -> mm_interconnect_0:intel

```

Figure 14. Screenshot of the SOPC System module.

Signal Tap Logic Analyzer

This lab involves using the SignalTap Logic Analyzer to observe and debug the internal signals within the FPGA while the SDRAM system is operating. A configuration file for the SignalTap was designed using Quartus Prime and was connected to the system clock signal. Internal signals such as the chipselect signal, address signal, writedata signal, data_out signal, write_n signal, reset_n signal, and wait request signal.

A trigger condition was set to when chipselect =1, indicating whenever the SDRAM controller was being accessed by the processor. The design was recompiled and programmed onto the FPGA after configuring the Signal Tap file.

The test program was created using the NIOSV processor through the RiscFree IDE. The memory test program was run, and the Signal Tap Logic Analyzer captured the behaviour of the transactions between the Avalon bus. The signal tap waveform shows the processor reading and writing commands to the SDRAM controller, including address selection, data transfer, and control signals. The read and write operation was used to verify the correct operations occurring between the processor and SDRAM.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 20 21:35:13 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab4
Top-level Entity Name	lab4
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	4,370 / 32,070 (14 %)
Total registers	5768
Total pins	57 / 457 (12 %)
Total virtual pins	0
Total block memory bits	1,463,424 / 4,065,280 (36 %)
Total DSP Blocks	3 / 87 (3 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	0 / 4 (0 %)

Figure 15. Compilation report of the XXX module.

Table 4. Summary for Components for the XXX in Lab 4.

Feature	Quantity
Total number of logic elements	4370
Total number of registers	5768
Total number of pins	57
Max number of logic elements that can fit on FPGA	32,070

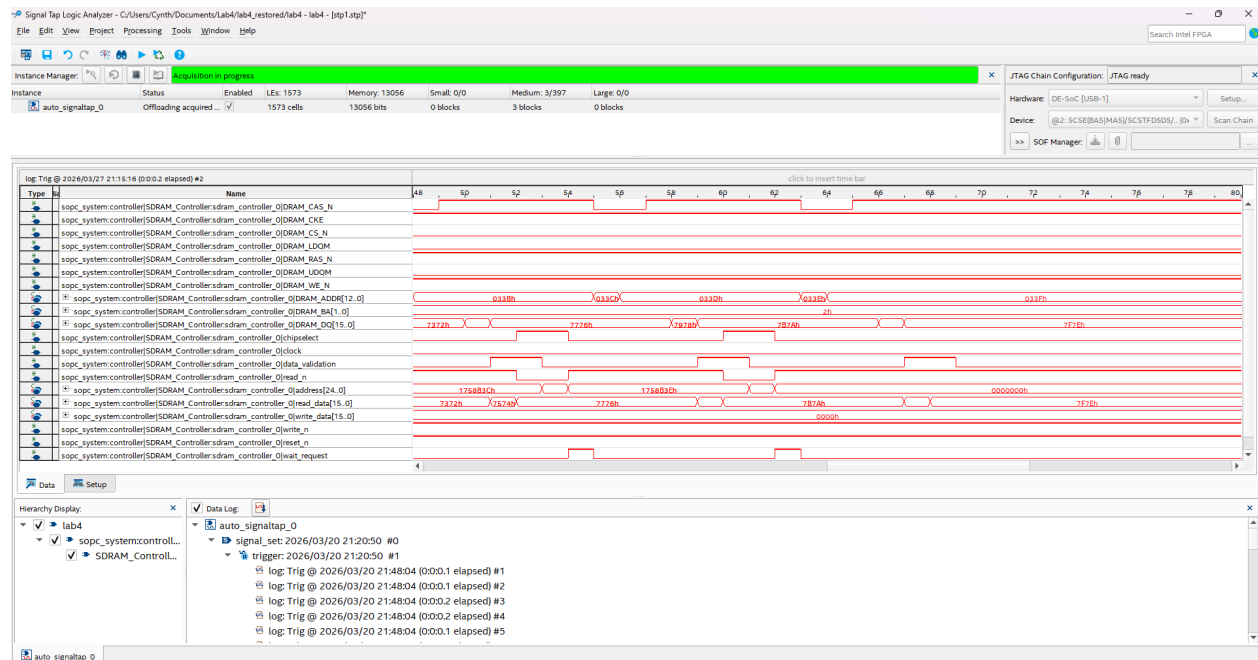


Figure 16. Screenshot of the Signal Tap Logic Analyzer.

log: Trig @ 2026/03/20 21:57:21 (0:0:0.1 elapsed)		
Type	lia	Name
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_CAS_N
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_CKE
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_CS_N
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_LDQM
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_RAS_N
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_UDQM
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_WE_N
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_ADDR[12..0]
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_BA[1..0]
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 DRAM_DQ[15..0]
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 chipselect
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 clock
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 data_validation
* -		sopc_system:controller SDRAM_Controller:sdram_controller_0 read_n
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 address[24..0]
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 read_data[15..0]
C +		⊕ sopc_system:controller SDRAM_Controller:sdram_controller_0 write_data[15..0]

Figure 17. Screenshot of the XXX module.

C Program for Memory Test

The C program used in the Nios V system is designed to test the communication between the processor and SDRAM memory using the SDRAM controller. The program is created and compiled using the Ashling RiscFree. In the program, the Nios V processor writes data to specific SDRAM addresses and then reads back that data to ensure that memory read/write operations are functioning correctly. In addition, different data types, such as char, short, and int, are used to read and write memory using different data sizes, which represent 8-bit, 16-bit, and 32-bit data types. These data types are used to ensure that the SDRAM controller is able to support different data sizes and that the processor is able to read and write data to SDRAM memory correctly. The testing is performed to ensure that the SDRAM controller is able to support different data sizes and that the processor is able to read and write data to SDRAM memory correctly.

```
tag uart terminal [Program] C:\intelFPGA\23.1std\quartus\bin64\juart-terminal.exe [pid: 16372] (Mar 20, 2026, 8:15:39 p.m.) [pid: 16372]
juart-terminal: connected to hardware target using JTAG UART on cable
juart-terminal: "DE-SoC [USB-1]", device 2, instance 0
juart-terminal: (Use the IDE stop button or Ctrl-C to terminate)

Hello from MT3TB4 Group 6!

Testing 33554432 words out of Maximum 33554432 words On SDRAM.....

the sizeof char, short, int in bytes are: 1, 2, 4

writing chars.....
testing chars.....

Testing Char: the total numbers of error is : 0

writing short.....
testing short.....

Testing Short: the total numbers of error is : 0

writing integer.....
testing integer.....

Testing int: the total numbers of error is : 0

Memory test complete. # Char errors: 0. # Short errors: 0. # int errors: 0.
Memory test passed.
```

Figure 18. Screenshot of the Nios Console output.

As shown in the juart-terminal above, the characters, shorts, and integers were written and read back from the SDRAM. The memory test was performed by comparing the write and retrieved values, and since there were zero errors, it confirms that the SDRAM controller and the Avalon bus are functioning correctly, and the data is stored and accessed accurately.

```
hello_world.c x
#include "system.h"
#include <stdio.h>

#define BASE SDRAM_CONTROLLER_0_BASE
#define SDRAM_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2
//the size of the SDRAM on DE1-SoC is 64MB =67108864 Byte (SDRAM_SPAN)

#define Testing_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2 //5
//To test part of the memory, for example, to test the first 1/10 of the memory:
//#define Testing_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/20

int main() {
    printf("Hello from MT3TB4 Group 6!\n\n\n");
    printf("Testing %d words out of Maximum %d words On SDRAM.....\n\n\n", Testing_MAXNUM_WORDS, SDRAM_MAXNUM_WORDS);

    int i;
    int char_err_num = 0, short_err_num = 0, int_err_num = 0;

    char * char_ptr;
    char aChar;
    short *short_ptr;
    short aShort;
    int *int_ptr;
    int aInt;

    int charsize, shortsize, intsize;
    charsize = sizeof(aChar);
    shortsize = sizeof(aShort);
    intsize = sizeof(aInt);

    printf("the sizeof char, short, int in bytes are: %d, %d, %d\n", charsize, shortsize, intsize);

    //-----TEST CHAR-----
    while(1){
        printf("\n writing chars.....\n");
    }
```

Figure 19. Screenshot of the module for the memory test.

Top Level Module

The top-level module is the main structural module that connects the Nios V system and the SDRAM controller to the hardware on the DE1-SoC board. In this module, an instance of the SOPC system created by Platform Designer is created and connected to the board's clocks and reset signals. The SDRAM controller module has also been created and connected to both the Avalon interface of the Nios system and the external pins on the FPGA board that connect to the memory chip. The signals required to connect to the memory chip, including address lines, bank address lines, control signals, and data lines, are also tied to the corresponding pins on the FPGA board. This module connects everything to ensure proper communication between the processor, controller, and memory. Once all connections were made, the design was compiled using Quartus and then implemented on the FPGA board.

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Fri Mar 27 05:02:15 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab4
Top-level Entity Name	lab4
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	4,370 / 32,070 (14 %)
Total registers	5768
Total pins	57 / 457 (12 %)
Total virtual pins	0
Total block memory bits	1,463,424 / 4,065,280 (36 %)
Total DSP Blocks	3 / 87 (3 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	0 / 4 (0 %)

Figure 20. Compilation report of the Top Level Module.

Table 5. Summary for Components for the Top Level Module in Lab 4.

Feature	Quantity
Total number of logic elements	4370
Total number of registers	5768
Total number of pins	57
Max number of logic elements that can fit on FPGA	32,070

```

*****
*
* Module:      Lab4
* Description:
*   This module is the top level module of MT3TB4 Lab 4
*
*****/

module lab4 (
input      CLOCK_50,
input [0:0] KEY,
input [7:0] SW,           //for reset
output [7:0] LEDR,

// Bidirectionals
inout [15:0]  DRAM_DQ,

// Outputs
output [12:0]  DRAM_ADDR,
output [1:0]  DRAM_BA,
output        DRAM_LDQM, //data mask; when it is low, the DQ is valid for reading and writing.
output        DRAM_UDQM,
output        DRAM_RAS_N,
output        DRAM_CAS_N,
output        DRAM_CLK,
output        DRAM_CKE,
output        DRAM_WE_N,
output        DRAM_CS_N
);

// Internal wires

```

Figure 21. Screenshot of the Top-level lab4.v module.

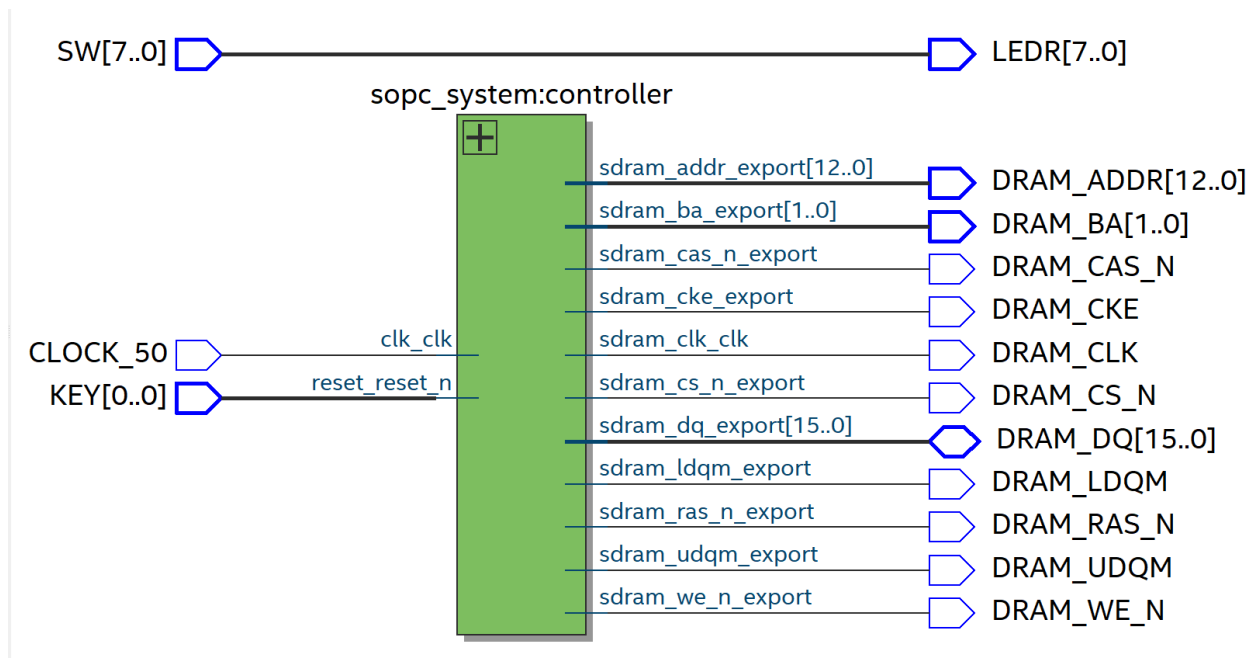


Figure 22. RTL view of the circuit.

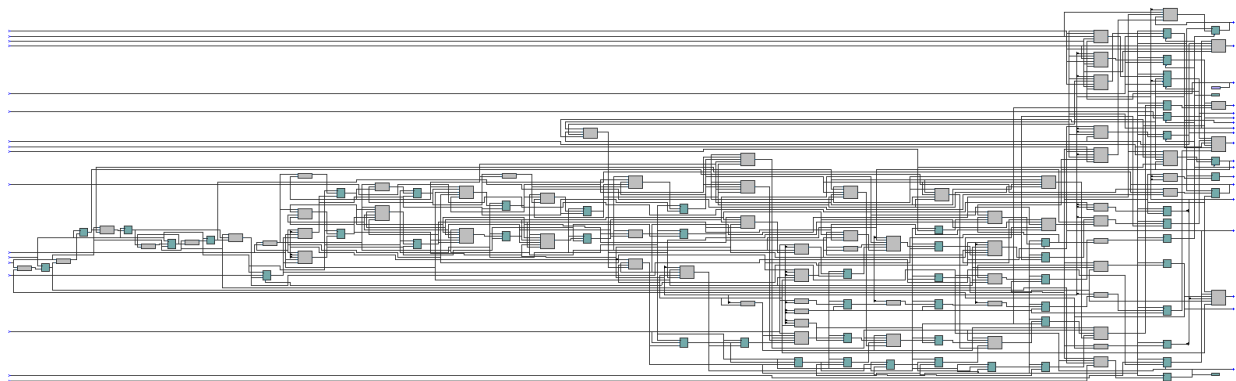


Figure 23. Technology map viewer (post-mapping) of the circuit.

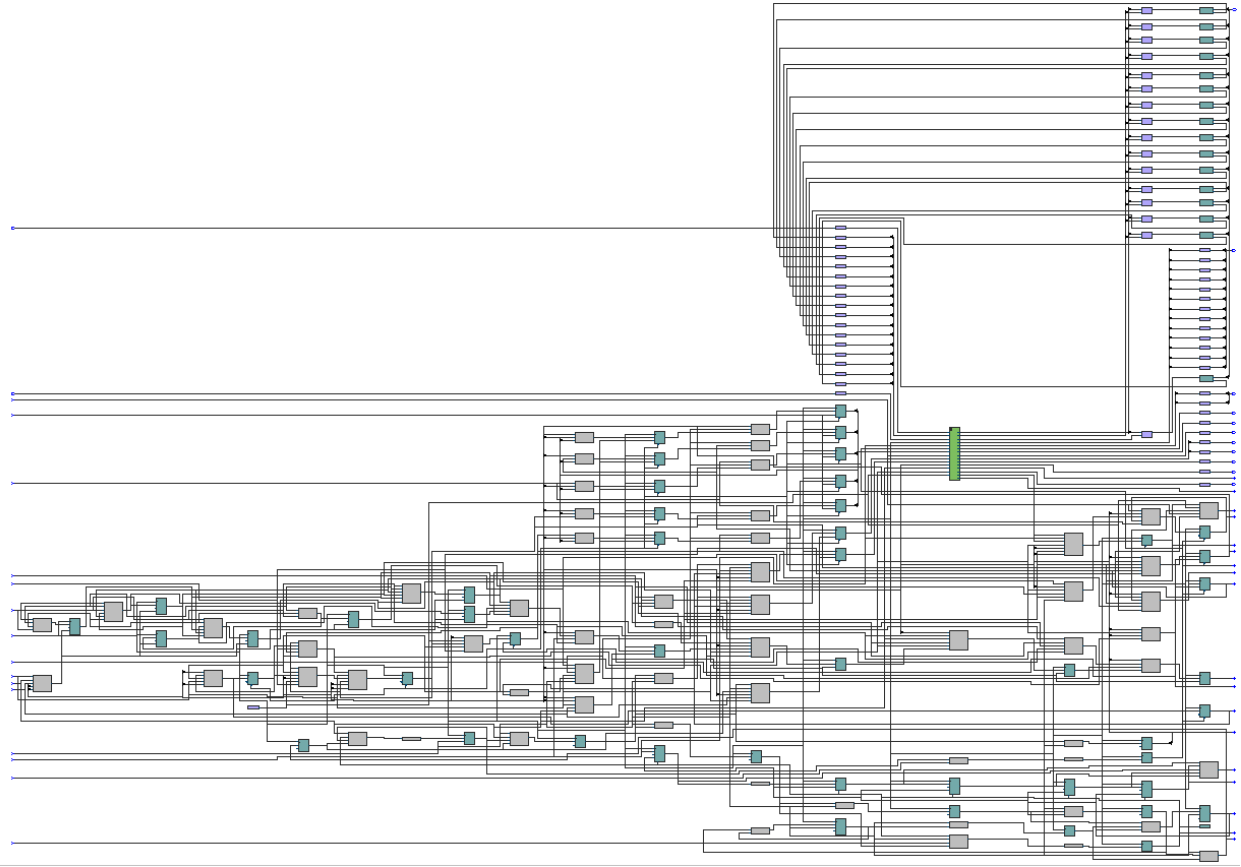


Figure 24. Technology map viewer (post-fitting) of the circuit.

Top View - Wire Bond

Cyclone V - 5CSEMA5F31C6

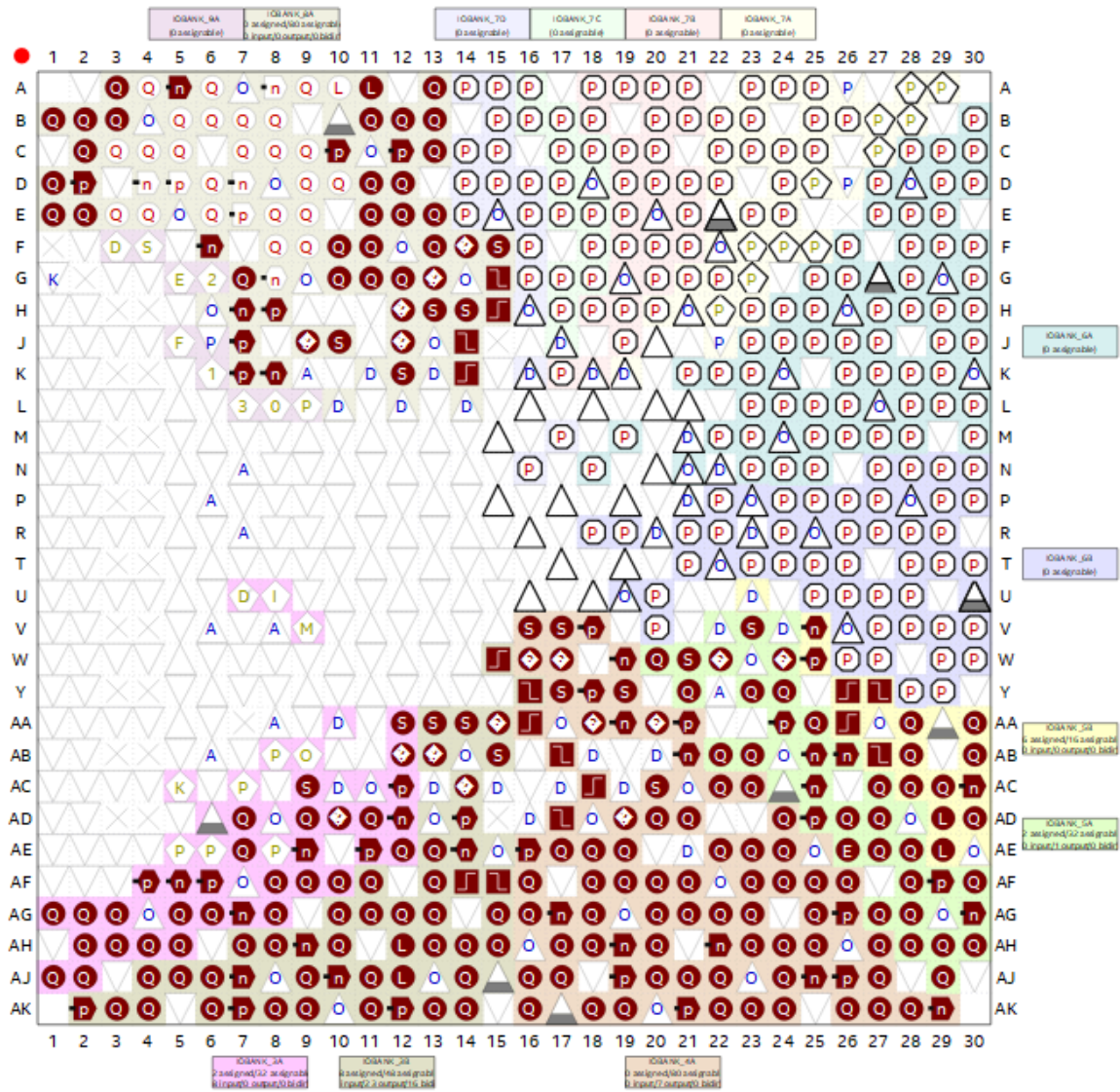


Figure 25a. Pin planner image.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
CLOCK_50	Input	PIN_AF14	3B	B3B_NO	PIN_AF14	2.5 V		12mA (default)	
DRAM_ADDR[12]	Output	PIN_AJ14	3B	B3B_NO	PIN_AJ14	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[11]	Output	PIN_AH13	3B	B3B_NO	PIN_AH13	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[10]	Output	PIN_AG12	3B	B3B_NO	PIN_AG12	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[9]	Output	PIN_AG13	3B	B3B_NO	PIN_AG13	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[8]	Output	PIN_AH15	3B	B3B_NO	PIN_AH15	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[7]	Output	PIN_AF15	3B	B3B_NO	PIN_AF15	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[6]	Output	PIN_AD14	3B	B3B_NO	PIN_AD14	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[5]	Output	PIN_AC14	3B	B3B_NO	PIN_AC14	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[4]	Output	PIN_AB15	3B	B3B_NO	PIN_AB15	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[3]	Output	PIN_AE14	3B	B3B_NO	PIN_AE14	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[2]	Output	PIN_AG15	3B	B3B_NO	PIN_AG15	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[1]	Output	PIN_AH14	3B	B3B_NO	PIN_AH14	2.5 V		12mA (default)	1 (default)
DRAM_ADDR[0]	Output	PIN_AK14	3B	B3B_NO	PIN_AK14	2.5 V		12mA (default)	1 (default)
DRAM_BA[1]	Output	PIN_AJ12	3B	B3B_NO	PIN_AJ12	2.5 V		12mA (default)	1 (default)
DRAM_BA[0]	Output	PIN_AF13	3B	B3B_NO	PIN_AF13	2.5 V		12mA (default)	1 (default)
DRAM_CAS_N	Output	PIN_AF11	3B	B3B_NO	PIN_AF11	2.5 V		12mA (default)	1 (default)
DRAM_CKE	Output	PIN_AK13	3B	B3B_NO	PIN_AK13	2.5 V		12mA (default)	1 (default)
DRAM_CLK	Output	PIN_AH12	3B	B3B_NO	PIN_AH12	2.5 V		12mA (default)	1 (default)
DRAM_CS_N	Output	PIN_AG11	3B	B3B_NO	PIN_AG11	2.5 V		12mA (default)	1 (default)
DRAM_DQ[15]	Bidir	PIN_AJ5	3B	B3B_NO	PIN_AJ5	2.5 V		12mA (default)	1 (default)
DRAM_DQ[14]	Bidir	PIN_AJ6	3B	B3B_NO	PIN_AJ6	2.5 V		12mA (default)	1 (default)
DRAM_DQ[13]	Bidir	PIN_AH7	3B	B3B_NO	PIN_AH7	2.5 V		12mA (default)	1 (default)
DRAM_DQ[12]	Bidir	PIN_AH8	3B	B3B_NO	PIN_AH8	2.5 V		12mA (default)	1 (default)
DRAM_DQ[11]	Bidir	PIN_AH9	3B	B3B_NO	PIN_AH9	2.5 V		12mA (default)	1 (default)
DRAM_DQ[10]	Bidir	PIN_AJ9	3B	B3B_NO	PIN_AJ9	2.5 V		12mA (default)	1 (default)
DRAM_DQ[9]	Bidir	PIN_AJ10	3B	B3B_NO	PIN_AJ10	2.5 V		12mA (default)	1 (default)
DRAM_DQ[8]	Bidir	PIN_AH10	3B	B3B_NO	PIN_AH10	2.5 V		12mA (default)	1 (default)
DRAM_DQ[7]	Bidir	PIN_AJ11	3B	B3B_NO	PIN_AJ11	2.5 V		12mA (default)	1 (default)
DRAM_DQ[6]	Bidir	PIN_AK11	3B	B3B_NO	PIN_AK11	2.5 V		12mA (default)	1 (default)
DRAM_DQ[5]	Bidir	PIN_AG10	3B	B3B_NO	PIN_AG10	2.5 V		12mA (default)	1 (default)
DRAM_DQ[4]	Bidir	PIN_AK9	3B	B3B_NO	PIN_AK9	2.5 V		12mA (default)	1 (default)
DRAM_DQ[3]	Bidir	PIN_AK8	3B	B3B_NO	PIN_AK8	2.5 V		12mA (default)	1 (default)
DRAM_DQ[2]	Bidir	PIN_AK7	3B	B3B_NO	PIN_AK7	2.5 V		12mA (default)	1 (default)
DRAM_DQ[1]	Bidir	PIN_AJ7	3B	B3B_NO	PIN_AJ7	2.5 V		12mA (default)	1 (default)
DRAM_DQ[0]	Bidir	PIN_AK6	3B	B3B_NO	PIN_AK6	2.5 V		12mA (default)	1 (default)
DRAM_LDQM	Output	PIN_AB13	3B	B3B_NO	PIN_AB13	2.5 V		12mA (default)	1 (default)
DRAM_RAS_N	Output	PIN_AE13	3B	B3B_NO	PIN_AE13	2.5 V		12mA (default)	1 (default)
DRAM_UDQM	Output	PIN_AK12	3B	B3B_NO	PIN_AK12	2.5 V		12mA (default)	1 (default)
DRAM_WE_N	Output	PIN_AA13	3B	B3B_NO	PIN_AA13	2.5 V		12mA (default)	1 (default)
KEY[0]	Input	PIN_AA14	3B	B3B_NO	PIN_AA14	2.5 V		12mA (default)	
LEDR[7]	Output	PIN_W20	5A	B5A_NO	PIN_W20	2.5 V		12mA (default)	1 (default)
LEDR[6]	Output	PIN_Y19	4A	B4A_NO	PIN_Y19	2.5 V		12mA (default)	1 (default)
LEDR[5]	Output	PIN_W19	4A	B4A_NO	PIN_W19	2.5 V		12mA (default)	1 (default)
LEDR[4]	Output	PIN_W17	4A	B4A_NO	PIN_W17	2.5 V		12mA (default)	1 (default)
LEDR[3]	Output	PIN_V18	4A	B4A_NO	PIN_V18	2.5 V		12mA (default)	1 (default)
LEDR[2]	Output	PIN_V17	4A	B4A_NO	PIN_V17	2.5 V		12mA (default)	1 (default)
LEDR[1]	Output	PIN_W16	4A	B4A_NO	PIN_W16	2.5 V		12mA (default)	1 (default)
LEDR[0]	Output	PIN_V16	4A	B4A_NO	PIN_V16	2.5 V		12mA (default)	1 (default)
SW[7]	Input	PIN_AC9	3A	B3A_NO	PIN_AC9	2.5 V		12mA (default)	
SW[6]	Input	PIN_AE11	3A	B3A_NO	PIN_AE11	2.5 V		12mA (default)	
SW[5]	Input	PIN_AD12	3A	B3A_NO	PIN_AD12	2.5 V		12mA (default)	
SW[4]	Input	PIN_AD11	3A	B3A_NO	PIN_AD11	2.5 V		12mA (default)	
SW[3]	Input	PIN_AF10	3A	B3A_NO	PIN_AF10	2.5 V		12mA (default)	
SW[2]	Input	PIN_AF9	3A	B3A_NO	PIN_AF9	2.5 V		12mA (default)	
SW[1]	Input	PIN_AC12	3A	B3A_NO	PIN_AC12	2.5 V		12mA (default)	
SW[0]	Input	PIN_AB12	3A	B3A_NO	PIN_AB12	2.5 V		12mA (default)	

Figure 25b. Pin planner table.

Code

SDRAM Controller

```
/******  
*                               *  
* Module:    SDRAM_Controller  *  
* Description:                               *  
*   This module is used for the sram controller for MT3TB4 Lab 4   *  
*                               *  
*****/  
  
`timescale 1ns / 1ps  
  
module SDRAM_Controller ( //the ports sequence follows the sequence of wires in the diagram  
in the lab manual  
    //signals through Avalon Interface:  
    input          clock,  
    input          reset_n,  
    input          chipselect,  
    input          write_n,  
    input          read_n,  
    input  [1:0]  byteenable_n,  
    input  [24:0] address, //2's power of 25 is 33,554,432 that is 32M. (16 bits  
word, that makes capable for 64MBytes)  
    input  [15:0] write_data,  
    output [15:0] read_data,  
    output          wait_request,  
    output          data_validation,  
  
    //signals between Controller and SDRAM:  
  
    inout  [15:0] DRAM_DQ,  
    output [12:0] DRAM_ADDR,  
    output [1:0]  DRAM_BA,  
    //    output          DRAM_CLK,  
    output          DRAM_CKE,  
    output          DRAM_LDQM,  
    output          DRAM_UDQM,  
    output          DRAM_WE_N,  
    output          DRAM_CAS_N,  
    output          DRAM_RAS_N,  
    output          DRAM_CS_N  
);
```

```

wire                clock_wire/*synthesis keep*/;
wire                reset_n_wire/*synthesis keep*/;
wire                chipselect_wire/*synthesis keep*/;
wire                write_n_wire/*synthesis keep*/;
wire                read_n_wire/*synthesis keep*/;
wire [1:0]          byteenable_n_wire/*synthesis keep*/;
wire [24:0] address_wire/*synthesis keep*/; //2's power of 25 is 33,554,432
that is 32M. (16 bits word, that makes capable for 64MBytes)
wire [15:0] write_data_wire/*synthesis keep*/;
wire                Wait_request_wire/*synthesis keep*/;
wire                data_validation_wire/*synthesis keep*/;

reg [15:0] read_data_reg/*synthesis preserve*/;
//wire [15:0] read_data_wire/*synthesis keep*/;

//signals between Controller and SDRAM:

wire [15:0] DRAM_DQ_wire/*synthesis keep*/;
wire [12:0] DRAM_ADDR_wire/*synthesis keep*/;
wire [1:0]   DRAM_BA_wire/*synthesis keep*/;

wire                DRAM_CKE_wire/*synthesis keep*/;

wire [1:0] DRAM_DQM_wire/*synthesis keep*/; //higher bit for UDQM
, lower bit for LDQM
wire                DRAM_WE_N_wire/*synthesis keep*/;
wire                DRAM_CAS_N_wire/*synthesis keep*/;
wire                DRAM_RAS_N_wire/*synthesis keep*/;
wire                DRAM_CS_N_wire/*synthesis keep*/;

wire [15:0] m_data_wire;
wire                output_enable_wire;

DE1_SoC_QSYS_sdram my_sdram (
    // inputs:
    .az_addr(address_wire),
    .az_be_n(byteenable_n_wire),
    .az_cs(chipselect_wire),
    .az_data(write_data_wire),
    .az_rd_n(read_n_wire),
    .az_wr_n(write_n_wire),
    .clk(clock_wire),
    .reset_n(reset_n_wire),

```

```

// outputs:
//.za_data(read_data_wire), //can not get read_data from here.
.za_valid(data_validation_wire),
.za_waitrequest(Wait_request_wire),
.zs_addr(DRAM_ADDR_wire),
.zs_ba(DRAM_BA_wire),
.zs_cas_n(DRAM_CAS_N_wire),
.zs_cke(DRAM_CKE_wire),
.zs_cs_n(DRAM_CS_N_wire),
//.zs_dq(DRAM_DQ_wire),
.zs_dqm(DRAM_DQM_wire),
.zs_ras_n(DRAM_RAS_N_wire),
.zs_we_n(DRAM_WE_N_wire),

.output_enable(output_enable_wire),

.internal_m_data(m_data_wire)
);

//=====Make connections =====

assign          DRAM_DQ=output_enable_wire?m_data_wire:{16{1'bz}} ;

//this way works , get data from za_data port does not works.
always @(posedge clock or negedge reset_n) //must be synchronize with the
clock!
begin
    if (reset_n == 0)
        read_data_reg<= 0;
    else
        read_data_reg <= DRAM_DQ;
end

assign          clock_wire=clock;
assign          reset_n_wire=reset_n;
assign          chipselect_wire=chipselect;
assign          write_n_wire=write_n;
assign          read_n_wire=read_n;
assign          byteenable_n_wire=byteenable_n;
assign          address_wire=address; //2's power of 25 is 33,554,432
that is 32M. (16 bits word, that makes capable for 64MBytes)
assign          write_data_wire=write_data;

```

```

        //output signals

        assign          read_data=read_data_reg;
        //assign          read_data=read_data_wire; //can not get
read_data from the za_data port

        assign          wait_request=Wait_request_wire;
        assign          data_validation=data_validation_wire;

        //signals between Controller and SDRAM:

        assign          DRAM_UDQM=DRAM_DQM_wire[1];          //higher bit for
UDQM , lower bit for LDQM
        assign          DRAM_LDQM=DRAM_DQM_wire[0];

        // ***** More necessary connections that YOUR NEED TO MAKE for Lab 4
project *****

        assign          DRAM_ADDR=DRAM_ADDR_wire;
        assign          DRAM_BA   =DRAM_BA_wire;
        assign          DRAM_CKE   =DRAM_CKE_wire;
        assign          DRAM_WE_N=DRAM_WE_N_wire;
        assign          DRAM_CAS_N=DRAM_CAS_N_wire;
        assign          DRAM_RAS_N=DRAM_RAS_N_wire;
        assign          DRAM_CS_N=DRAM_CS_N_wire;

endmodule

```

C Program for Memory Test

```

#include "system.h"
#include <stdio.h>

#define BASE_SDRAM_CONTROLLER_0_BASE
#define SDRAM_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2
    //the size of the SDRAM on DE1-SoC is 64MB =67108864 Byte (SDRAM_SPAN)

#define Testing_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2 //5

```

```

//To test part of the memory, for example, to test the first 1/10 of the memory:
//#define Testing_MAXNUM_WORDS    SDRAM_CONTROLLER_0_SPAN/20

int main() {
    printf("Hello from MT3TB4 Group 6!\n\n\n");
    printf("Testing %d words out of Maximum %d words On SDRAM.....\n\n\n",
Testing_MAXNUM_WORDS, SDRAM_MAXNUM_WORDS);

    int i;
    int char_err_num = 0, short_err_num = 0, int_err_num = 0;

    char * char_ptr;
    char aChar;
    short *short_ptr;
    short aShort;
    int *int_ptr;
    int aInt;

    int charsize, shortsize, intsize;
    charsize = sizeof(aChar);
    shortsize = sizeof(aShort);
    intsize = sizeof(aInt);

    printf("the sizeof char, short, int in bytes are: %d, %d, %d\n", charsize,
shortsize, intsize);

    //-----TEST CHAR-----
    while(1){
        printf("\n writing chars.....\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS * 2; i++) {
            *(char*) (BASE + i) = i % 128; // to be safe, use 128 rather than 256
        }

        printf("\n testing chars.....\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS * 2; i++) {
            if (*(char*) (BASE + i) != i % 128) { // or .....(char)i,    if not i%128

```

```

        char_err_num++;
    }
}
printf("\nTesting Char: the total numbers of error is : %i\n",
char_err_num);

//-----TEST SHORT-----

printf(" \n writing short.....\n");
for (i = 1; i <= Testing_MAXNUM_WORDS; i++) {
    *(short*) (BASE + i * 2) = i % 32767; // short, uses two bytes
}

printf(" \n testing short.....\n");
for (i = 1; i <= Testing_MAXNUM_WORDS; i++) {
    if (*(short*) (BASE + i*2) != i % 32767) { // or .....(short)i, if not
i%32767
        short_err_num++;
    }
}
printf("\nTesting Short: the total numbers of error is : %i\n",
short_err_num);

//-----TEST INT -----

printf(" \n writing integer.....\n");
for (i = 1; i <= Testing_MAXNUM_WORDS / 2; i++) {
    *(int*) (BASE + i * 4) = i; // int, use 4 bytes
}

printf(" \n testing integer.....\n");
for (i = 1; i <= Testing_MAXNUM_WORDS / 2; i++) {
    if (*(int*) (BASE + i*4) != i) { // or .....(int)i, if not i
        int_err_num++;
    }
}
printf("\nTesting int: the total numbers of error is : %i\n", int_err_num);

```

```

    printf("\nMemory test complete. # Char errors: %i. # Short errors: %i. # int
errors: %i.", char_err_num, short_err_num, int_err_num);

    if(!char_err_num && !short_err_num && !int_err_num == 1){
        printf("\nMemory test passed.");
    }else{
        printf("\nMemory test failed.");
    }
}
return 0;
}

```

Top Level Module

```

/*****
*
*
* Module:      Lab4
* Description:
*   This module is the top level module of MT3TB4 Lab 4
*
* *****/

module lab4 (
    input          CLOCK_50,
    input [0:0]    KEY,
    input [7:0] SW,    //for reset
    output [7:0] LEDR,

    // Bidirectionals
    inout [15:0]    DRAM_DQ,

    // Outputs

    output [12:0] DRAM_ADDR,
    output [1:0]  DRAM_BA,
    output          DRAM_LDQM, //data mask; when it is low, the DQ is valid
    for reading and writing.
    output          DRAM_UDQM,
    output          DRAM_RAS_N,

```

```

output          DRAM_CAS_N,
output          DRAM_CLK,
output          DRAM_CKE,
output          DRAM_WE_N,
output          DRAM_CS_N

```

```

);

```

```

// Internal Wires

```

```

assign LEDR=SW;

```

```

//Instantiate your socp_system module generated by Platform Designer.

```

```

socp_system controller (
    // example ports
    .clk_clk(CLOCK_50),          //      clk.clk
    .reset_reset_n(KEY[0]),      //      reset.reset_n
    .sdram_addr_export(DRAM_ADDR), // .sdram_addr_export
    .sdram_ba_export(DRAM_BA),    //sdram_ba_export
    .sdram_cas_n_export(DRAM_CAS_N), // sdram_cas_n.export
    .sdram_cke_export(DRAM_CKE),  // sdram_cke.export
    .sdram_clk_clk(DRAM_CLK),     // sdram_clk.clk
    .sdram_cs_n_export(DRAM_CS_N), // sdram_cs_n.export
    .sdram_dq_export(DRAM_DQ),    // sdram_dq.export
    .sdram_ldqm_export(DRAM_LDQM), // sdram_ldqm.export
    .sdram_ras_n_export(DRAM_RAS_N), // sdram_ras_n.export
    .sdram_udqm_export(DRAM_UDQM), // sdram_udqm.export
    .sdram_we_n_export(DRAM_WE_N) // sdram_we_n.export

);

```

```

endmodule

```